COSTRA — Full Test Cases

Review presentation reference • Copy-paste SQL inputs • Expected outputs • Error cases • Cost-calculation walkthrough

How to run a test

Open the project in VS Code and run `python3 main.py`. Choose **4** to enter custom SQL, then paste one supported SELECT statement as one line. Choose **3** for the full demo, **5** for cost calculation, **6** for review-wise progress and **7** for the intentional unknown-column error.

Best live-demo sequence

- **TC02:** custom SELECT with filter and descending sort.
- **TC08:** join + table aliases.
- **TC07:** GROUP BY + HAVING.
- **TC21:** full pipeline, estimated plan comparison and actual rows.
- **TC22:** unknown-column semantic rejection.
- **TC35:** cost calculation and lowest-cost decision.

Test case details

TC01 — Basic SELECT + LIMIT (Valid query)

Input SQL

```
SELECT name, salary FROM Employee LIMIT 3;
```

Expected result: PASS; 3 rows: Arjun | 85000; Neha | 72000; Riya | 48000.

Purpose: Shows lexer, parser, semantic analysis, planning and execution on a simple query.

TC02 — WHERE filter + descending order (Valid query)

Input SQL

```
SELECT name, salary FROM Employee WHERE salary > 80000 ORDER BY salary DESC;
```

Expected result: PASS; Ishita | 91000; Arjun | 85000. Estimated path uses an index for the salary predicate.

Purpose: Good short custom-input demo.

TC03 — ORDER BY + LIMIT (Valid query)

Input SQL

```
SELECT name FROM Employee ORDER BY salary DESC LIMIT 3;
```

Expected result: PASS; Ishita; Arjun; Aman.

Purpose: Shows ordering and row limiting.

TC04 — DISTINCT values (Valid query)

Input SQL

```
SELECT DISTINCT dept_id FROM Employee ORDER BY dept_id;
```

Expected result: PASS; department IDs 1, 2, 3, 4.

Purpose: Shows duplicate elimination.

TC05 — Scalar aggregate (Valid query)

Input SQL

```
SELECT COUNT(*) AS employee_count FROM Employee;
```

Expected result: PASS; employee_count = 8.

Purpose: Shows aggregate function handling.

TC06 — GROUP BY counts (Valid query)

Input SQL

```
SELECT dept_id, COUNT(*) AS n FROM Employee GROUP BY dept_id ORDER BY dept_id;
```

Expected result: PASS; (1, 3), (2, 2), (3, 2), (4, 1).

Purpose: Good aggregation example.

TC07 — GROUP BY + HAVING + AVG (Valid query)

Input SQL

```
SELECT dept_id, AVG(salary) AS avg_salary FROM Employee GROUP BY dept_id HAVING AVG(salary) > 70000 ORDER BY
avg_salary DESC;
```

Expected result: PASS; dept 4 = 91000.0; dept 1 = 78333.3333 (approximately).

Purpose: Demonstrates aggregate filtering after grouping.

TC08 — INNER JOIN (Valid query)

Input SQL

```
SELECT e.name, d.name AS department FROM Employee e JOIN Department d ON e.dept_id = d.id ORDER BY e.id LIMIT 4;
```

Expected result: PASS; Arjun | Engineering; Neha | Engineering; Riya | HR; Kabir | Sales.

Purpose: Shows alias resolution and join handling.

TC09 — IN predicate (Valid query)

Input SQL

```
SELECT name FROM Employee WHERE dept_id IN (1, 4) ORDER BY name;
```

Expected result: PASS; Aman; Arjun; Ishita; Neha.

Purpose: Demonstrates IN list parsing/execution.

TC10 — BETWEEN predicate (Valid query)

Input SQL

```
SELECT name, salary FROM Employee WHERE salary BETWEEN 60000 AND 80000 ORDER BY salary;
```

Expected result: PASS; Kabir | 65000; Priya | 69000; Neha | 72000; Aman | 78000.

Purpose: Shows range filtering.

TC11 — LIKE pattern (Valid query)

Input SQL

```
SELECT name FROM Employee WHERE name LIKE 'A%' ORDER BY name;
```

Expected result: PASS; Aman; Arjun.

Purpose: Shows string pattern matching.

TC12 — IS NOT NULL (Valid query)

Input SQL

```
SELECT name FROM Project WHERE manager_id IS NOT NULL ORDER BY name;
```

Expected result: PASS; AI Platform; Analytics; CRM; ERP; Mobile App.

Purpose: Shows null predicate support.

TC13 — Empty result / IS NULL (Valid query)

Input SQL

```
SELECT name FROM Project WHERE manager_id IS NULL;
```

Expected result: PASS; 0 rows returned (the seeded projects all have manager IDs).

Purpose: Useful no-results edge case; zero rows is not an error.

TC14 — Arithmetic expression (Valid query)

Input SQL

```
SELECT name, salary + 1000 AS adjusted_salary FROM Employee WHERE id = 1;
```

Expected result: PASS; Arjun | 86000.

Purpose: Shows expression parsing and SQL execution.

TC15 — LEFT JOIN (Valid query)

Input SQL

```
SELECT e.name, d.name AS department FROM Employee e LEFT JOIN Department d ON e.dept_id = d.id ORDER BY e.id
LIMIT 3;
```

Expected result: PASS; Arjun | Engineering; Neha | Engineering; Aman | Engineering.

Purpose: Shows outer-join syntax; optimizer preserves its order.

TC16 — COUNT(DISTINCT ...) (Valid query)

Input SQL

```
SELECT COUNT(DISTINCT dept_id) AS departments FROM Employee;
```

Expected result: PASS; departments = 4.

Purpose: Shows aggregate DISTINCT.

TC17 — NOT IN (Valid query)

Input SQL

```
SELECT name FROM Employee WHERE dept_id NOT IN (1, 2) ORDER BY name;
```

Expected result: PASS; Ishita; Kabir; Rahul.

Purpose: Negative membership condition.

TC18 — NOT BETWEEN (Valid query)

Input SQL

```
SELECT name FROM Employee WHERE salary NOT BETWEEN 60000 AND 80000 ORDER BY name;
```

Expected result: PASS; Arjun; Ishita; Rahul; Riya.

Purpose: Negative range predicate.

TC19 — NOT LIKE (Valid query)

Input SQL

```
SELECT name FROM Employee WHERE name NOT LIKE 'A%' ORDER BY name;
```

Expected result: PASS; Ishita; Kabir; Neha; Priya; Rahul; Riya.

Purpose: Negative string-pattern predicate.

TC20 — AND/OR precedence (Valid query)

Input SQL

```
SELECT name FROM Employee WHERE dept_id = 1 OR dept_id = 2 AND salary > 70000 ORDER BY name;
```

Expected result: PASS; Aman; Arjun; Neha. AND binds more tightly than OR.

Purpose: Useful parser precedence explanation.

TC21 — Full compiler/optimizer demo (Valid query)

Input SQL

```
SELECT d.name AS department, COUNT(e.id) AS employee_count, AVG(e.salary) AS average_salary FROM Employee e JOIN
Department d ON e.dept_id = d.id JOIN Assignment a ON e.id = a.employee_id JOIN Project p ON a.project_id = p.id
WHERE e.salary > 60000 AND p.budget > 100000 GROUP BY d.name HAVING AVG(e.salary) > 65000 ORDER BY AVG(e.salary)
DESC LIMIT 5;
```

Expected result: PASS; Finance | 1 | 91000.0; Engineering | 5 | 79600.0; HR | 1 | 69000.0. 3 rows. 6 candidate plans; lowest estimate 115222.63 model units (Plan 1 and Plan 2 tie).

Purpose: Best end-to-end demonstration of all implemented stages.

TC22 — Unknown column (Expected semantic rejection)

Input SQL

```
SELECT e.unknown_column FROM Employee e;
```

Expected result: REJECT at SEMANTIC ANALYSIS: Unknown column: e.unknown_column. No SQL is executed.

Purpose: Menu option 7 demonstrates an unknown-column rejection.

TC23 — Unknown table (Expected semantic rejection)

Input SQL

```
SELECT name FROM Staff;
```

Expected result: REJECT at SEMANTIC ANALYSIS: Unknown table: Staff; Unknown column: name.

Purpose: Shows schema-aware name validation.

TC24 — Unknown alias (Expected semantic rejection)

Input SQL

```
SELECT x.name FROM Employee e;
```

Expected result: REJECT at SEMANTIC ANALYSIS: Unknown table or alias: x.

Purpose: Shows alias checking.

TC25 — Ambiguous unqualified column (Expected semantic rejection)

Input SQL

```
SELECT id FROM Employee e JOIN Department d ON e.dept_id = d.id;
```

Expected result: REJECT at SEMANTIC ANALYSIS: Ambiguous column: id (matches e.id, d.id).

Purpose: Shows ambiguity detection.

TC26 — Missing projection (Expected syntax rejection)

Input SQL

```
SELECT FROM Employee;
```

Expected result: REJECT at SYNTAX ANALYSIS: unexpected FROM at line 1, column 8.

Purpose: Shows syntax error location.

TC27 — Illegal character (Expected lexical rejection)

Input SQL

```
SELECT @ FROM Employee;
```

Expected result: REJECT at LEXICAL ANALYSIS: unexpected character '@' at line 1, column 8.

Purpose: Shows lexical error handling.

TC28 — Aggregate inside WHERE (Expected semantic rejection)

Input SQL

```
SELECT e.name FROM Employee e WHERE AVG(e.salary) > 50000;
```

Expected result: REJECT at SEMANTIC ANALYSIS: Aggregate function AVG is not allowed in this clause.

Purpose: Shows clause-specific semantic validation.

TC29 — Invalid GROUP BY projection (Expected semantic rejection)

Input SQL

```
SELECT e.name, COUNT(*) FROM Employee e GROUP BY e.dept_id;
```

Expected result: REJECT at SEMANTIC ANALYSIS: Selected column e.name must appear in GROUP BY or be aggregated.

Purpose: Shows grouping rule validation.

TC30 — Non-integer LIMIT (Expected syntax rejection)

Input SQL

```
SELECT name FROM Employee LIMIT 2.5;
```

Expected result: REJECT at SYNTAX ANALYSIS: LIMIT must be a non-negative integer.

Purpose: Shows boundary/type constraint for LIMIT.

TC31 — Empty IN list (Expected syntax rejection)

Input SQL

```
SELECT name FROM Employee WHERE id IN ();
```

Expected result: REJECT at SYNTAX ANALYSIS: invalid empty IN list.

Purpose: Shows malformed predicate rejection.

TC32 — Unsupported function (Expected semantic rejection)

Input SQL

```
SELECT explode(e.salary) FROM Employee e;
```

Expected result: REJECT at SEMANTIC ANALYSIS: Unsupported function: EXPLODE.

Purpose: Shows supported-function validation.

TC33 — AVG on text column (Expected semantic rejection)

Input SQL

```
SELECT AVG(e.name) FROM Employee e;
```

Expected result: REJECT at SEMANTIC ANALYSIS: AVG requires a numeric column.

Purpose: Shows basic aggregate argument-type checking.

TC34 — Multiple statements (Expected syntax rejection)

Input SQL

```
SELECT name FROM Employee; SELECT name FROM Department;
```

Expected result: REJECT at SYNTAX ANALYSIS: multiple statements are not supported.

Purpose: Confirms one input must be one supported SELECT.

TC35 — Cost calculation and best estimated plan

Command: `python3 main.py --cost-demo` or choose menu option 5.

Worked SQL input

```
SELECT e.name FROM Employee e WHERE e.salary > 60000;
```

Using the current synthetic model: Employee input estimate N = 10,000; selectivity = 0.72; estimated output ≈ 7,200 rows.

Full scan + predicate = 10,000 + (0.20 × 10,000) = **12,000**.

Index scan = 2 × log2(10,000) + (0.60 × 7,200) = **4,346.58**.

The index scan is cheaper under the model. For the full multi-join query, Plans 1 and 2 tie at **115,222.63**; Plans 3 and 4 tie at 121,702.63; Plans 5 and 6 tie at 123,862.63 model units. The program returns the first minimum candidate.

Important: these are synthetic model units, not milliseconds. COSTRA's selected estimate is not passed to SQLite; SQLite chooses its own execution plan independently.

TC36 — Review-wise demonstration commands

Goal	Command / menu	Expected demonstration
Review 1	`--review1-demo` / option 1	Problem, objectives, scope, architecture
Review 2	`--review2-demo` / option 2	Lexer, parser/AST, semantics, IR, sample execution
Review 3	`--demo` / option 3	Full pipeline, optimizer estimates and SQLite output
Own SQL	Menu option 4	Paste a valid SELECT query

Goal	Command / menu	Expected demonstration
Cost	`--cost-demo` / option 5	Worked formula, candidate comparison, minimum cost
Progress	`--progress-demo` / option 6	Review 1 → Review 2 → Review 3
Error	`--error-demo` / option 7	Reject unknown column before execution

Automated verification

Run `python3 -m unittest -v`. The latest checked suite completed with **52 tests passing**. Re-run on the submission machine and capture fresh terminal output as evidence.

The execution backend uses a small in-memory fixture: 8 employees, 4 departments, 5 projects and 8 assignments. The optimizer uses separate synthetic statistics (Employee 10,000 rows, Project 1,200 rows, Assignment 30,000 rows); these are not collected from SQLite.